GENERACIÓN DE DOCUMENTOS PDF EN VUE

1. Objetivo

Incorporar a una aplicación desarrollada con Vue 3 una funcionalidad que permite generar documentos PDF con información procedente de la base de datos. Los documentos podrán descargarse, guardarse o imprimirse. Por ejemplo:

- Listado de doctores.
- Listado de pacientes.
- Listados filtrados por especialidad u otros criterios.
- Ficha individual de un doctor o paciente.

2. Bibliotecas recomendada

Para empezar, se recomienda ***jsPDF*** junto con ***jsPDF-AutoTable***. **jsPDF** permite crear documentos PDF mediante ***JavaScript*** y ***AutoTable*** facilita la creación de tablas que pueden distribuirse automáticamente entre varias páginas.

Biblioteca	Función	Uso habitual
jsPDF	Crear el documento PDF.	Informes y fichas personalizadas.
jsPDF-AutoTable	Generar tablas a partir de cabeceras y filas.	Listados con varias columnas y páginas.
pdfmake (alternativa)	Crear documentos estructurados mediante definiciones.	Informes con encabezados, pies y tablas.

Documentación oficial:

- jsPDF: https://github.com/parallax/jsPDF
- jsPDF-AutoTable: https://github.com/simonbengtsson/jsPDF-AutoTable
- pdfmake: https://pdfmake.github.io/docs/

3. Instalación

Desde la terminal, en el directorio del proyecto Vue, ejecutamos:

```
npm install jspdf jspdf-autotable
```

La instalación añade las dependencias necesarias al proyecto.

4. Generar un PDF con el listado de doctores

Paso 1. Importar las bibliotecas

```
import { jsPDF } from "jspdf";
import autoTable from "jspdf-autotable";
```

Paso 2. Crear la función de generación

El siguiente ejemplo supone que doctores es un ref de Vue que contiene los doctores recuperados de la base de datos.

```
const generarPDFDoctores = () => {
 const doc = new jsPDF();

 // Título
 doc.setFontSize(18);
 doc.text("Listado de doctores", 14, 20);

 // Fecha de generación
 doc.setFontSize(10);
 doc.text(
  `Fecha: ${new Date().toLocaleDateString("es-ES")}`,
  14,
  28
 );

 // Tabla
```

```
autoTable(doc, {
 startY: 35,
 head: [[
  "ID", "Apellidos", "Nombre", "Móvil", "Especialidad"
 ]],
 body: doctores.value.map((doctor) => [
  doctor.iddoc,
  doctor.apeldoc,
  doctor.nomedoc,
  doctor.movildoc,
  doctor.espedoc
 ]),
 styles: {
  fontSize: 9,
  cellPadding: 3
 },
 headStyles: {
  fillColor: [30, 201, 164]
 }
});

// Descargar el documento
doc.save("listado_doctores.pdf");
};
```

1. La función **crea** el documento
2. **Añade** el título y la fecha
3. **Genera** la tabla y **descarga** el PDF.

AutoTable **calcula la distribución** de las filas y añade páginas cuando sea necesario.

Nota: en ***<script setup>,*** si doctores está ***declarado como ref([])***, se accede a su contenido mediante ***doctores.value.***

5. Añadir un botón en Vue

Podemos colocar un botón **junto al título del listado**:

```
<div class="cabecera-listado">
 <h4>Listaxe de Doctores</h4>
// usar un icono report
 <button
  type="button"
  @click="generarPDFDoctores"
  :disabled="doctores.length === 0"
 >
  Exportar PDF
 </button>
</div>
```

CSS opcional:

```
.cabecera-listado {
 display: flex;
 justify-content: space-between;
 align-items: center;
 gap: 1rem;
}

.cabecera-listado button {
 padding: 0.5rem 1rem;
 cursor: pointer;
}
```

Al pulsar el botón se descarga el PDF. El botón queda deshabilitado si no hay doctores cargados.

6. Crear una función genérica reutilizable. Opcional

Para **no repetir el código en cada componente**, podemos crear una función que **reciba el título, las cabeceras, las filas y el nombre del archivo.**

```
const generarPDF = (titulo, cabeceras, filas, nombreArchivo) => {
 const doc = new jsPDF();

 doc.setFontSize(18);
 doc.text(titulo, 14, 20);

 doc.setFontSize(10);
 doc.text(
  `Fecha: ${new Date().toLocaleDateString("es-ES")}`,
  14,
  28
 );

 autoTable(doc, {
  startY: 35,
  head: [cabeceras],
  body: filas,
  styles: {
   fontSize: 9,
   cellPadding: 3
  },
  headStyles: {
   fillColor: [30, 201, 164]
  }
 });

 doc.save(nombreArchivo);
};
```

Ejemplo: exportar doctores

```
const exportarDoctores = () => {
 generarPDF(
  "Listado de doctores",
  ["ID", "Apellidos", "Nombre", "Móvil", "Especialidad"],
  doctores.value.map(d => [
   d.iddoc,
   d.apeldoc,
   d.nomedoc,
   d.movildoc,
   d.espedoc
  ]),
  "listado_doctores.pdf"
 );
};
```

Ejemplo: exportar pacientes

Suponiendo que **pacientes** es un **ref** con los campos *dnipac, apelpac, nomepac y movilpac:*

```
const exportarPacientes = () => {
 generarPDF(
  "Listado de pacientes",
  ["DNI", "Apellidos", "Nombre", "Móvil"],
  pacientes.value.map(p => [
   p.dnipac,
   p.apelpac,
   p.nomepac,
   p.movilpac
  ]),
  "listado_pacientes.pdf"
 );
};
```

Adapta los nombres de los campos a los que realmente existan en cada entidad. Los componentes solo tienen que preparar los datos; la lógica de creación del PDF se mantiene en una función común.

7. Mejoras posibles

A. Encabezado y pie de página

Añadir el nombre de la aplicación, la fecha de generación y la numeración de páginas. AutoTable ofrece mecanismos para personalizar encabezados y pies de página.

B. Orientación de la página

Vertical, adecuada para tablas estrechas:

```
const doc = new jsPDF("portrait");
```

Horizontal, útil para tablas con muchas columnas:

```
const doc = new jsPDF("landscape");
```

C. Exportar resultados filtrados

Si el listado está filtrado por especialidad, búsqueda u otro criterio, podemos pasar a la función los resultados filtrados en lugar de todos los registros.

D. Ficha individual

Se puede crear una plantilla específica para una ficha de doctor o paciente, con el título y los datos distribuidos en campos en lugar de una tabla.

E. Organizar el código

```
src/
├── api/
│   ├── doctores.js
│   └── pacientes.js
├── utils/
│   └── pdf.js
└── components/
    ├── XestionDoctores.vue
    └── XestionPacientes.vue
```

La función genérica de generación se guardaría en ***utils/pdf.js.*** Los componentes la importarían para exportar sus respectivos datos.

8. Consideraciones importantes

- **Caracteres especiales:** comprobar que tildes, ñ y otros caracteres se representan correctamente. Para fuentes personalizadas, puede ser necesario incorporarlas al PDF.
- **Listados extensos**: probar la distribución entre páginas con un número elevado de registros.
- **Privacidad:** los datos de pacientes son información personal y pueden incluir datos de salud especialmente protegidos. Exportar solo los campos necesarios y restringir el acceso.
- **Datos actualizados:** verificar que el array contiene todos los registros que se quieren exportar y no solo los de la página actual.
- **Validación:** probar con listados vacíos, nombres largos y caracteres especiales.

9. Plan de implementación

Fase	Funcionalidad	Dificultad
1	Exportar el listado completo de doctores a PDF.	Baja
2	Crear una función genérica para doctores, pacientes y otras entidades.	Media
3	Añadir fichas individuales, filtros, encabezados y numeración de páginas.	Media

Recomendación final: empezar con jsPDF y jsPDF-AutoTable para el listado de doctores. Cuando funcione, extraer la función común a ***utils/pdf.js*** y reutilizarla en el resto de componentes. Esto permite añadir la exportación sin alterar la lógica existente de consulta, inserción, modificación o eliminación de registros en MySQL.